Date .....
S M T W TH F S

Operating system.

CRYSTAL

Operating system:- Operating system is system software that manages computer hardware and software resources and provide services to application.
An OS act as a manager between the user, application and computer hardware.

Components of computer system:-

1) **Hardware:-** Hardware include the physical part of the computer such as CPU, RAM, Storage, Input and output devices. Hardware provide the basic computing.
2) **Operating system:-** The OS controls and coordinates how hardware is used by different programs and users. It decides how resources are shared.
3) **Application programs:-** Application programs used to perform specific task such as Word processor, Web browser, compiler, Games.
4) **User:-** User are people that request work from the computer. ("The OS kernel run with special hardware privileges allowing it to manage important system resources")

Two views of OS:- An operating system can be understood in two ways:-
1) **Resources Manager:-** The OS manages and distributes computer resources among different programs. These resources

Date .....
S M T W TH F S

CRYSTAL

Goals:- Effiziency, fairness, no conflict

include CPU time, RAM, Disk space, Input / output devices.

2) **Extended machine:-** The OS hides complicated hardware details and provide easier way for programs or user to work with computer. Goals:- Convenience, Simplicity, portability.

Goals of an OS:-

1) **Convenience:-** The OS make a computer easy to use. It provides user-friendly interfaces, simple file handling and command that help user perform tasks without understanding the complicated details of hardware.

2) **Effiziency:-** The OS tries to use CPU time, memory and other devices effectively. It help programs run smoothly and reduce the waste of computer resources.

3) **Ability to evolve:-** An OS should support new features, hardware and updates without unnecessarily breaking existing programs.

Core functions of OS:-

1) **Process management:-** An Process management is a program that is currently running. The OS create processes, decide when each process get CPU time, End processes when they finish.

2) **Memory management:-** The OS manages RAM and decides how memory is allocated to different programs. It also help protect the memory used by one process from unauthorized access by another.

CamScanner

Date .....
S M T W TH F S

CRYSTAL

3) **File Management:-** The OS organize files and folders and provides way to save, open, find and delete data

4) **Storage and Input/output:-** The OS manages storage devices and input/output devices with the help of device drivers.

5) **Security and protection:-** The OS controls access to files, resources and system operations. It uses users, permissions, and isolation to protect the system.

6) **Networking and user interface:-** The OS provide networking services so computer can communicate. It also provides interface through which user interact with the system

Kernel, Shell and system programs:-

1) **Kernel:-** The kernel is core part of OS. It remain in memory while the OS is running and manage important system operations. Its responsibilities include managing processes, managing memory, controlling hardware devices, handling system calls.

2) **Shell:-** The shell allows user to communicate with the operating system. It can be command interpreter or in broder sense part of user interface. Bash in Linux Powershell in windows

3) **System programs:-** System programs are utilies that help users and developers perform common task. like file manager, Compiler.

4) **Device Drivers:-** Device drivers are software components that help the OS communicate with specific hardware

Date .....
S M T W TH F S

devices.

5) **System Call**:- System calls provide a way for application programs to request services from the kernel.

Evolution of OS:-

1) **1940s – 1950s**: "NO operating system" Computers were operated more directly by people and operators. Programs used machine language and punch cards were used to provide instructions and data.

2) **1950s – 1960s**:- "Batch system" Similar jobs were grouped into batches and processed one after another.

3) **1960s – 1970s**:- "Multiprogramming and time-sharing" Time sharing allowed multiple user to interact with a computer by rapidly switching CPU time between their tasks. UNIX began in 1969.

4) **"1980s – 1990s"**:- "Personal computers." Computer became common for individuals. OS included MS-DOS, Windows, Mac OS and Linux which first appeared in 1991.

5) **~~1990s~~ 2000s – present**:- "Mobile, Cloud, AI." OS system support Smartphones, cloud servers, containers.

Batch vs. multiprogramming vs time share.

Feature	Batch	Multiprogramming	time-share
Working	Run jobs in group.	Keeps several jobs in memory.	Shares CPU time in small slices.
User-interaction	None during execution	Very little	Interactive
CPU use	Can be low	High	High
Response	Slow	Moderate	Fast
Example	Early mainframing	Early UNIX.	Modern Linux

Date .....
S M T W TH F S

CRYSTAL

Types of operating system:-

Batch:- Run jobs without user-interaction **Eg:-** Early mainframes.

Time sharing / Multitasking:- Shares CPU time among tasks **E.g:-** Linux, Windows.

Real-time:- Responds within required time-limits **E.g:-** robots.

Distributed:- Multiple computers work together. **E.g:-** in cloud cluster.

Mobile:- Design for smartphones. **E.g:-** Android and ios.

Embedded:- Designed for specific devices. **E.g:-** Kettles and smart TV.

Real-time OS : Hard vs. Soft.

Feature	Hard	Soft.
Deadline	Must be missed	Occassional days are allowed
Failure	May be dangerous	Usually reduce quality.
Examples	Airbags, Pacemaker.	Video streaming, Online gaming.
Main goal	Meet critical deadline	Balance speed and timing.

Ways to classify OS:-

Basis	Categories	Example
Users	Single user, multi user	MS-DOS, UNIX.
Tasks	Single tasking, multitasking	MS-DOS, Windows.
Processors	Uniprocessor, multiprocessor.	Old PCs, modern-multi core systems.
Interface.	CLI, GUI, Touch	Bash, Windows, desktop, IOS
Architecture.	32 bit, 64 bit.	Different system architecture
Source	Open source, Proprietary.	Linux, Windows.

CamScanner

Date .....
S M T W TH F S

CRYSTAL

Popular OS today:-

1) **Linux** : Servers, Cloud computing and AI research.
2) **Windows**: Desktop computers, business and gaming
3) **Mac OS /iOS**: Apple computers and mobile devices.
4) **Android**:- Smartphones and mobile application

Linux vs Window vs Mac OS:-

Feature	Linux	Windows	Mac OS:
Kernel	Linux.	NT.	XNU.
License	Open-source.	proprietary.	proprietary
Main use	Servers, Cloud, AI.	Desktop, gaming	Development, creative work.
Shell	Bash, Zsh.	Powershell, CMD.	Zsh, Bash.
Cost	Generally free.	Often licensed.	Included with apple hardware?
AI/ML.	Strong support.	Good support.	Good support

What happen when you press power?

1) **Power ON**:- The computer start.
2) **BIOS/UEFI**:- Checks and prepares hardware.
3) **Bootloader**:- Helps load the OS.
4) **Kernel load**:- The kernel start managing the system.
5) **Init/services**:- Background service start.
6) **Login/Shell**:- The user can begin working.

How the OS handles a keypress: Interupts:

An interrupt is a signal that tells the CPU an event needs action from end

Date .....
S M T W TH F S

1) A key is pressed.
2) The keyboard controller signals the CPU.
3) The OS information is stored in a buffer.
4) The application receives the input.
Interrupts allow the OS to respond to devices efficiently.

Why OS matter for AI and IT student:-

1) **AI training and inference:-** AI training means teaching a model using data. Inference means using a trained model to make prediction. The OS manages CPU and memory resources and supports communication with GPU's through drivers and related software.

2) **Cloud and DevOps:-** Cloud services rely heavily on operating systems, particularly Linux servers. Tools such as Docker and Kubernetes use OS features to manage application and isolated workload.

3) **Debugging and performance:-** Understanding processes, memory and input/output operations help developers identify why a program is slow, freeze, runs out of memory.

4) **Security:-** Operating system provide permissions, process isolation and access controls that help protect data and application.

OS behind an AI training job:-

1) **Load data:-** Operating system reads dataset files from storage.

Date .....
S M T W TH F S

CRYSTAL

2) **Fill RAM**:- Memory manager allocate memory.

3) **Start process**:- Schedular give CPU time to training program.

4) **USE GPU**:- Drivers and related software help the program use GPU.

5) **Save model**:- The program saves check points to disk.

=> If RAM runs out the system may use disk space for virtual memory, making training much slower.

Slide 2:-

Date .....
S M T W TH F S

Operating System

CRYSTAL

Goals when designing an OS:- An operating system should meet several important goals:-

1) **Correctness:-** The OS should work as expected without errors in important operations.

2) **Efficiency:-** It should use CPU, memory and devices without unnecessary waste.

3) **Maintainability:-** Developers should be able to fix, update and improve it easily.

4) **Portability:-** It should run on different hardware with only a few changes.

5) **Security:-** It should protect data and processes from unauthorized access.

Approach 1: Simple / Monolithic - legacy structure:- In simple or monolithic - legacy structure the OS components do not have a clear separation into layers. Its structure includes application programs, resident system program, MS-DOS device drivers. The components are not clearly separated an error in one part can affect other parts and may crash the whole system.

Approach 2: Monolithic Kernel:- A monolithic kernel runs most major operating system services together as one large program in kernel mode. These process include process management, memory management, file system. Components can communicate directly with each other.

Advantages:- 1) Fast performance because components communicate directly.

Date .....
S M T W TH F S

CRYSTAL

2) Efficient execution of system operation

Disadvantages:- i) The kernel contain a large amount of code.

ii) Maintaing and updating the system can be difficult.

Approach 3 : Microkernel:- Microkernel keeps only essential services inside the kernel such as basic inter-process communication minimal memory management. Other service including file system, device drivers and networking run outside the kernel as user-space server.

Advantages: 1) Better stability and security.

2) A failure in one service is less likely to crash the whole OS.

Disadvantages:- 1) Communication between separate services require message passing which can reduce performance.

CamScanner

Slide #2

Layered and hybrid Kernel design:-

Operating system can also use layered, modular and hybrid design:-

Layered design:- The OS is divided into layer. Each layer uses services provided by layer directly below it. **e.g:-** The OS.

Modular design:- The kernel has core components and additional modules that can be loaded when needed. **e.g:-** Solaris

Hybrid design:- Combined feature of monolithic and microkernel designs to balance speed and modularity. **e.g:-** Windows NT.

Kernel architecture at a glance:-

Feature	Monolithic	Microkernel	Hybrid
Speed	Fast	Slow due to message passing	Fast
Stability	failure affect the whole kernel.	Better isolation of service.	High.
Kernel size	Very large.	Very small	Medium to large.
Extensibility	Harder	Easier	Moderate.
Example.	Linux, traditional UNIX	QNX, MINIX.	Window NT.

CamScanner

Date .....
S M T W TH F S

CRYSTAL

Dual mode operation: User mode vs. Kernel mode:-

User mode:-

- Application and most user program run in this mode.
- Programs have restricted access and cannot directly access hardware.
- They request operating system services through system calls.
- Example include web browser, Python script.

Kernel mode:-

- The operating system kernel run in this mode.
- It has full hardware access and can execute privileged instructions.
- It manages memory, devices and processes directly.

2) Hardware mode bit identifies which bit is currently active.

How a mode switch happens:- A mode switch occurs when a program needs an OS services that require kernel privileges.

1) User mode :- The application runs with restricted access.
2) System call / Interrupt :- The program request a service or an event occur.
3) Trap to kernel :- Control passes to kernel, and CPU switch to kernel mode.
4) Kernel Executes :- The OS performs the required operation.
5) Return to user :- The CPU switches back to user mode and application continues.

The switching happens many times per second and is usually invisible to user.

System call:- System call is a controlled way for a user program to request a service from the OS kernel.

Date .....
S M T W TH F S

CRYSTAL

System calls act like a menu of services provided by OS. Application usually access them through libraries of API.

Major categories of system call:-

Category	Purpose	Example.
Process Control	Create, end or wait for process.	fork(), exe(), exit().
File management	Open, read, write and close file	open(), read(), write().
Device management.	Control and access device.	ioctl(), read(), write().
Information Maintence.	Get or set system information.	getpid(), alarm(), time().
Communication.	Exchange data between process	pipe(), shmget(), send().
Protection.	Manage permissions and access	chmod(), umask().

Common Linux system calls you will meet:-

1) fork() :- Create new child process based on parent process.
2) exec() :- Replace the current process program with new program
3) wait() :- Allows a parent process to wait for child process to finish
4) open() / close() :- open and close files.
5) read() / write() :- Read data and write data.
6) mmap() :- Maps a file or device into memory

System call interface : API vs OS. Applications commonly use library functions or API to request operating system serves:-

Application:- Calls a library function such as `printf()` in C or `open()` in python

API / library:- The standard library such as `libc`, prepare the appropriate system call and its argument.

Date .....
S M T W TH F S

CRYSTAL

Kernel:- Perform the requested privileged operation and returns a result through the library to application.

The Shell: Your Window into OS:-

A shell is a command line interpreter that reads command entered by a user and ask the OS to execute them.

Learning the shell is useful because servers, cloud platforms and AI/ML pipeline are often controlled through command line tools. It can also be be faster and more powerful then clicking through graphical menus

Date .....
S M T W TH F S

Operating system.

CRYSTAL

Processes:- Process is a program that is currently running on computer. The operating system manages processes and give them the resource they need. Processes allow us to use different applications such as browser and text editor.

Why do we need processes:- Processes allow a computer to work with multiple programs at same time. Each process gets its own memory which help prevent one program from another. The OS manages CPU and memory usage and switches between processes they need.

Program vs. process:-

Program:- Program is a file stored on computer that contains instructions. It does not perform any work until it is executed.

Process:- Process is a program that is running. **e.g:-** Google chrome is a program but opening three separate chrome processes means the program is running in three process

Where a process lies in memory:- A process uses four main memory areas: Code, Data, Heap and stack. Code contain the programs instructions, while data stores global variables. Heaps stores memory requested while the program runs, and stack stores temporary information for function call

What is process doing:- Process moves through five states:- New, Ready, Running, Waiting and terminated. New means

Date .....
S M T W TH F S

CRYSTAL

the process is being created. Ready means it is waiting for CPU time. and Running means it is using the CPU. Waiting means it is waiting for an event while terminated means it is finished.

How a process changes state:- Process change its state according to what is happening during execution. New → Ready means the operating system accept it and Ready → Running means it gets the CPU. Running → Waiting happens when it needs input or a file while waiting → Ready happens when that task is completed. A process moves to terminated when it finishes.

PCB: The ID card of process:- PCB stands for process control Block. It is a data structure used by the OS to store important information about each process. It contain the process state, process ID, program counter, CPU registers, priority, memory information, CPU usage and open file. It works like an identification card for a process

Why the PCB matter:- The PCB helps the OS manage and control processes. When process stops running the OS saves its current information in PCB so the process can continue later from where it stopped. The PCB is created when a process starts and removed when its ends. All PCB together form process table, and Linux uses task_struct for its process stack.

CamScanner

Date .....
S M T W TH F S

CRYSTAL

Waiting line:- Queues are used to keep processes organized while they wait for resources. The Ready Queue contains processes waiting for CPU the Device Queue contains processes waiting for devices and the Job Queue contains all the process in the system. When a process finishes waiting for device it returns to Ready Deue.

The three Schedulers:- Schedular are the part of OS that decide how processes are managed.

Long-term:- Decide which new processes are allowed to enter the system.

Short-term:- Selects the next ready process to use the CPU.

Medium term:- Move processes between memory and disk when memory is full and bring them back later.

The OS tries to make good balance between processes that wait for Input/output and those that perform calculation.

Context Switch: changing the process on CPU:- Context Switch happens when the CPU changes from running one process to running another. The OS saves the current process information in its PCB and select the next process. It then loads the next process saved information so it can continue where they stopped. Too many content switch waste time because the CPU does not perform useful application work during the switching process.

Date .....
S M T W TH F S

CRYSTAL

When does a context switch happen:- A context switch can happen when a process allocated CPU time end or higher priority process becomes ready. It may also happen when a process waits for a file, disk or keyboard. Other causes include a device interrupt, a process finish.

Dispatcher and cost of switching:- The Dispatcher is the part of OS that gives control of the CPU to the selected process. It switch the context, returns the CPU to user mode when appropriate and start the process at the correct instruction. Dispatch Latency is the time needed to stop one process and start another. Switching also has a hidden cost because the CPU cache may contain data from the previous process.

Mode switch vs. context switch:- Mode switch change the CPU between user mode and kernel mode and it can happen while the same process continues running. Context switch changes the CPU from one process to another and takes more time. Context switch involves mode switching but a mode switch does not always require context switch

Making new processes:- A process can create another process. The process that creates a new one is called parent and the new process is called child. In Unix/Linux `fork()` creates a child process `exec()` replace the child

Date .....
S M T W TH F S

CRYSTAL

program with a new program and [wait()] allow the parent to wait for the child to finish. Every process has a unique process ID. Window uses [createprocess] to start a new program.

fork() step-by-step:- When a parent process call [fork()] the operating system creates a child process with new PID. Both process continue from instruction after [fork()]. The function return [0] to child. the child PID to parent and [-1] if creation fails. The child may use [exec()] to run another program while the parent may use [wait()] to wait for it. Two [fork()] calls in a row can create four processes and n forks can create 2^n processes under the usual assumption that every process executes both call successfully.

Ending a process:- A process ends when it finishes its work or call [exit()]. The OS then realse its resources such as memory and open file. A zombie is a process that has finished but whose parent has not yet collected its exit status. An orphan is a process whose parent finishes first the system process [init] adopt it.

Zombies and orphan:- Zombie:- A child process finishes but its parent has not called [wait] to collect its result. The child has ended but remains in process list until its status is collected.

Date .....
S M T W TH F S

CRYSTAL

Orphan:- A parent process finishes while its child is still running. The child is then adopted by the system process [init] which help manage it and clean up after it finishes.

Processes talking to Each other:- Processes sometimes needs to exchange information or work together. This is called inter-process communication. IPC allow processes to share information coordinate their work and divide large task into smaller parts. Two main methods are shared memory and message passing.

Shared memory:- Shared memory is common area of memory that two or more processes can access. Processes can read and write information in this area making communication fast.

Producer and customer example:- The producer add item to shared buffer and customer removes them. The [in] pointer show where to add an item, while [out] shows where to remove one. The buffer is empty when [in] equal [out] and full when next [in] equal out. If both process access the same buffer at same time incorrectly a race condition can occur.

Pipes and message:- Pipes allow one process to send its output to another process as input **e.g:-** ls | wc. An ordinary pipe usually provide one way communication between related processes. Named pipe work like a file and can be used by different processes. Communication can be blocking or unblocking.

CamScanner

Date .....
S M T W TH F S

Operating System.

CRYSTAL

Problem with process:- Creating a new process takes time because the OS must create a PCB and manage its memory. Switching between processes also require extra work and sharing data between separate processes can be difficult. When one programs need to perform multiple jobs, creating many processes may waste memory. so threads provide alternative.

Thread:- Thread is a single path of execution inside a process called a lightweight process. **E.g:-** Think of process as a kitchen and its threads as cooks: all cook share the same kitchen but perform different jobs.
Every process has one thread and a process with multiple thread is called multithreaded.

What thread share and keep separate:- Each thread has its own thread ID, program counter, register and stack. Threads belonging to the same process share code, data, heap memory, open files. Each thread need a separate stack for its own function call and local variable, while the shared heap allow thread to access common data.

Single thread vs. Many thread:- Single threaded process has one path of execution so if its thread waits for a file or another operation the whole program may have to wait. Multithread process has several path of execution that share the same memory if one thread waits another may

Date .....
S M T W TH F S

CRYSTAL

continue working and each thread has its own registers and stack.

Real example of thread:- In a web-browser, one thread may display a page another may download data and another may play a video. In a word processor thread can handle typing, spelling check and automatic saving. A web server can use threads to handle visitors, while an AI program may load data in one thread and train a model in another.

Benefits of threads:- Threads provide four main benefits- Responsiveness, Resource sharing, Economy, Scalability. Responsiveness allow a program to keep working while one task waits and resource sharing make it easier to access common memory and file. Threads are cheaper to create and switch than processes and they can run on different CPU cores.

Process vs. thread:- A process is generally slower and more expensive to create and switch, while thread is lighter and cheaper. Processes have separate memory, whereas threads share the memory of their process and can communicate through shared data. If one independent process crashes, other processes are generally protected. but a serious thread failure may crash the whole process.

CamScanner

Date .....
S M T W TH F S

CRYSTAL

Concurrency vs. parallelism:- Concurrency means multiple task make progress during the same period often by switching quickly between tasks on one CPU core. Parallelism means multiple tasks actually excute at same time on different CPU core. **E.g:-** one cook switching between two dishes demonstrates concurrency while two cooks preparing separate dishes simultaneously demonstrate parallelism.

Challenges of multicore programming:- Multicore programming requires identifying tasks that can run independently and balancing work between CPU core. Data must be divided carefully so threads do not interface with one another and some tasks may depend on the result of other tasks. Testing and debugging are also difficult because multiple threads can execut in different order.

Data parallelism vs Task parallelism:- Data parallelism means performing the same operation on different portion of data such as using four threads to calculate the sum of four parts of large list.
Task parallelism means performing different jobs at the same time such as using one thread to sort, another to search and another to print.
The main difference is work is divided by data or by task.

Date .....
S M T W TH F S

CRYSTAL

Amdahl's law:- Amdahl's law explains that the serial part of program limits how much faster it can becoming when more CPU core are added. Its formula is $\text{Speedup} = \frac{1}{S + \frac{1-S}{N}}$ where S is the serial fraction and N is the number of core. If 25% of program is serial two cores provide a speedup of 1.6 times while four cores provide approximately 2.29 times the maximum speedup is 4 times

User thread vs. Kernel thread:- User threads are managed by a thread library within a program making them generally faster to create and manage In traditional model described in slides blocking one user thread may block the whole process
Kernel threads are managed and scheduled by the OS can run on different CPU cores and allow other threads to continue when one blocks; however they are generally more expensive to create.

Multithreading model:- The three models are:-
i) many-to-one ii) one-to-one iii) many-to-many
In many-to-one many user threads map to one kernel thread.
In one-to-one each user thread maps to its own kernel thread.

Date .....
S M T W TH F S

CRYSTAL

In many-to-many, many user threads map to a smaller or equal number of kernel threads, providing a flexible connection between user level and kernel-level thread.

Comparing the three model:- Many-to-one is fast and simple but a blocking call may block all threads and true parallelism is not possible. One-to-one supports true parallelism and reduce blocking interference but many kernel thread require additional resources. Many-to-many support flexible thread mapping but is harder to implement and is described as rarely used today.

Thread Libraries:- Thread library provides functions that programmers use to create and manage thread. Pthreads is the POSIX thread standard for Linux and Unix-Window provide its own thread API, Java has built-in thread support managed by JVM and python provide the threading module for creating and managing threads.

Python: Threading vs. Multiprocessing:- Python threading uses multiple threads within the same process and is generally suitable for waiting tasks such as file and network operation.

Multiprocessing uses separate processes with separate memory and can perform CPU-heavy calculations in parallel

Date .....
S M T W TH F S

CRYSTAL

across multiple cores.

Creating threads :- Two ways :- Asynchronous execution occurs when a parent thread creates another thread and continues its own work without waiting for the new thread to finish.
Synchronous execution occurs when the parent waits for the created thread to finish before continuing. The `join()` operation is used to wait for thread to complete its work.

Threads pool :- Thread pool is a collection of threads created in advance to handle tasks. Instead of creating a new thread for every request the program assign an available thread returns to pool after completing the task. Thread pools improve response time, reduce repeated thread-creation costs and controls the number of thread.

Threads states and control block :- Threads have five basic states : new, ready, running, waiting and terminated.
A thread control block (TCB) stores the thread ID, program counter, registers, stack pointer and thread state. The slides also explains that linux uses the `clone()` system call to create threads each thread as a task.

Date .....
S M T W TH F S

Operating System.

CRYSTAL

CPU scheduling:- CPU scheduling is a process by which the OS decides which process will use the CPU next. Many processes may need the CPU at the same time but one CPU core can run only one process at a time.

Why is CPU scheduling needed:- The main goal of multiprogramming is to keep the CPU busy as much as possible. When a process waits for an input/output operation the OS gives the CPU to another ready process instead of leaving it idle. The CPU scheduler decides which ready process should run next.

CPU burst and I/O burst:- A process usually works in a cycle: it uses the CPU for some time, wait for an input/output operation and then uses the CPU again. This cycle continues until the process finishes.

I/O-bound process:- Has many short CPU bursts.

CPU-bound process:- Has a few long bursts.

CPU scheduler and Ready queue:- The CPU scheduler selects one process from the ready queue whenever the CPU needs a new process. The ready queue may be arranged according to arrival time, burst length, priority or other rules. It is not always first-in, first-out. Each queue entry represents a Process Control block. and the dispatcher give the CPU to selected process.

CamScanner

Date .....
S M T W TH F S

CRYSTAL

Preemptive vs. non-preemptive Scheduling:-

Non-preemptive:- Once a process gets the CPU it continues until it finishes or wait for I/O. It is simple but gives the OS less control over a running process.

Preemptive:- The OS can take the CPU away from a running process such as when its time expire or higher-priority process arrives. It is commonly used in modern operating system.

Dispatcher:- The dispatcher gives control of CPU to the process selected by the CPU schedular. Its main job are switching the process context, switching to user mode. Dispatch latency is the time required to stop one process and start another so, the dispatcher must work quickly.

CPU scheduling Criteria:-

Criteria	Simple meaning	Goal
CPU utilization	Percentage of time the CPU is busy	Maximize
Throughput	Number of processes completed per unit of time	Maximize.
Turnaround time.	Total time from arrival to completion	Minimize
Waiting time	Time spent waiting in ready queue.	Minimize.
Response time	Time from arrival until the process first gets the CPU	Minimize.

Important CPU Scheduling formulas:-

- Turnaround time = Completion time − Arrival time
- Waiting time = Turnaround time − Burst time
- Response time = First CPU start time − Arrival time.

Date .....
S M T W TH F S

CRYSTAL

- Average waiting time = Sum of waiting times ÷ Number of processes

FCFS (First come, first Served):- FCFS gives the CPU to the process that arrives first. It uses a FIFO queue and is non-preemptive scheduling algorithum It is simple and easy to implement but short processes may wait a long time behind longer processes.

SJF (Shortest Job first):- SJF selects the process with the smallest next CPU burst. It gives the minimum average waiting time when the required burst lengths are known making it optimal for this creation. However the OS cannot know the next CPU burst exactly in advance and long processes may suffer from starvation. SJF can be non-preemptive or preemptive. Its preemptive version is called Shortest Remaining time First (SRTF).

Predicting the Next CPU Burst:- Because the OS cannot know the exact future CPU burst. it estimates it using previous burst times. The slides use exponential averaging which combines the last actual burst with previous estimate.

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

Here t_n is the last actual burst. τ_n is the privous estimate and α is the value between 0 and 1. if $\alpha=0$, the latest burst is ignored if $\alpha=1$, only the latest burst is used.

Date .....
S M T W TH F S

CRYSTAL

SRTF (Shortest Remaining time first):- SRTF is the preemptive version of SJF. If a new process arrives with a shorter remaining CPU time than the currently running process. the OS interrupts the current process and runs the new one

Priority Scheduling:- In priority scheduling each process has a priority number and the CPU is assigned to the highest priority process. a smaller priority number means higher priority. This algorithm can be preemptive or non-preemptive. Its main problem is starvation where low-priority processes may wait indefinitely. Aging solve this problem by gradually increasing a waiting process priority. SJF is a special case of priority scheduling in which shorter CPU bursts receive higher priority.

Round Robin (RR) Scheduling:- Round Robin is a preemptive scheduling algorithm mainly used in time-sharing system. Each process gets a small fixed amount of CPU time called the time quantum. usually 10-100 milliseconds. When its quantum expires the process goes to back of ready queue if it still needs CPU time

Effect of time Quantum Size:- The time quantum determines how long a process can run before the CPU may be switched to another process. A smaller quantum usually improves response time but increases context-switch overhead. A large quantum reduces

Date .....
S M T W TH F S

CRYSTAL

Switching overhead but may make the system less responsive. A good quantum should be much larger than the time needed for a context switch.

Multi-level feedback queue:- MLFQ allow processes to move between queues according to their CPU usage and waiting time. New processes enter the top queue with small time quantum: processes that use their entire quantum move to lower queues while processes waiting too long can move up through aging

Comparison of scheduling algorithm:-

algorithm	preemptive	Main advantage	disadvantage
FCFS	No	Simple	Convoy effect
SJF/SRTF	No/Yes	Least waiting time	Needs burst guess
Priority	Either	Important jobs first	Starvation, fixed by aging
Round Robin	Yes	Fair, good response	Depends on quantum
MLFQ	Yes	Flexible, adapts	Complex to tune

CPU Scheduling in Real operating System:-

Linux:- Uses fair-share scheduling approaches, to distribute CPU time among tasks.

Window:- Uses preemptive, priority-based scheduling with multiple priority tasks.

Symmetric multiprocessing:- In multiprocessing systems a each CPU schedules its own task.

Processor affinity:- keeps a process on same CPU when possible so it can reuse cached data.

CamScanner

Date .....
S M T W TH F S

Operating System.

CRYSTAL

Process Synchronization:- Process synchronization is the process of controlling how processes and threads access shared data. When two threads change the same data at same time the result may be incorrect. Synchronization makes them access shared resources in a controlled way so result remain correct

Why is synchronization needed:- Processes and threads sometimes share the same data or resources. If they update shared data simultaneously the data may become inconsistent because the CPU can switch between them during an operation. Synchronization makes sure that shared data is accessed safely and produces correct results.

Race condition:- A race condition occurs when the final result depends on the order in which processes or thread execute their operation. **e.g.:-** if a counter is 5 and one thread increases it while another decrease it the correct result should be 5. If their operations overlap one update may be lost, producing an incorrect result. Such as 4 or 6.

Critical section:- Critical section is the part of program where shared data or resources are accessed or modified. Only one process should be allowed inside critical section at a time when mutual exclusion is required. The four sections are:-

- Entry section:- Request permission to enter.
- Critical section:- Accesses or modifies shared data
- Exit section:- Releases access so others can enter.
- Remainder section:- Executes the remaining code.

Date .....
S M T W TH F S

CRYSTAL

Three rules of good critical-section solution:- A correct solution must satisfy three important conditions:

Mutual exclusion:- Only one process can execute its critical section at a time

Progress:- If a critical section is empty a process that wants to enter should be selected without unnecessary delay

Bounded waiting:- A process must not wait forever there must be a limit on how many times other processes can enter before

Peterson's solution:- Peterson's solution is a software method for solving the critical section problem for two processes. Each process uses a flag to indicate that it wants to enter and a shared turn variable to give the other process a chance. A process waits while the other process wants to enter and it is the other process turn. Although the solution satisfies the three rules under its assumption

Hardware Support for Synchronization:- Hardware provides method to protect shared data from simultaneous access. Disabling interrupts can work on a single CPU but it inefficient and unsuitable as a general solution for multiprocessor systems. Atomic instruction such as `test_and_set` and `compare_and_swap` perform operations as one individual steps preventing another process from interrupting the operation halfway through.

Lock using test_and_set:- A shared lock initially has the value `false` meaning it is available. A process repeatedly executes `test_and_set` until it obtains the lock which changes the value to `true`. Other processes keep trying until the lock

become available when the process finishes it sets the lock back to [false]. this repeated checking is called busy waiting and the lock is called spinlock

Mutex Locks:- A mutex lock is simple tool used to protect a critical section. The [acquire()] operation obtains the lock while [release()] frees it for another process. These operations must be atomic to work correctly. Spinlocks can waste CPU time while waiting so they are most useful when the expected time is very short.

Semaphores:- Semaphores is an integer variable used to control access to shared resources through two atomic operations. The [wait(s)] operation waits until the semaphore is positive and then decrease its value by one while [signal(s)] increase increase its value one.

Counting and binary Semaphore:- A counting semaphore can hold different integer value and is used to manage multiple identical resources. if there are three printers the semaphore can start at 3.

A binary semaphore has only two values 0 and 1. and can be used to control access to a shared resource in a way similar to mutex lock

Semaphores without busy waiting:- A semaphores can use a waiting queue instead of making processes repeatedly check it's value. If a process calls [wait] and no resource is available it is blocked and placed in queue. When another process call [signal] a waiting process can be awakened. This saves

Date .....
S M T W TH F S

CRYSTAL

CPU time while waiting, although, blocking and waking processes involve context-switching overhead.

Problems with semaphores:- Semaphores can cause problem if they are used incorrectly.

Deadlock: Two or more processes wait forever for resources held by one another.

Starvation: A process waits indefinitely because other processes keep getting the resource first.

Priority inversion:- A low-priority process holds a lock needed by a high-priority process. Priority inheritance can help by temporarily raising the lock holder's priority. This problem affected the Mars Pathfinder robot in 1997.

Classic Synchronization problems:- These describe three classic problems used to study synchronization problem. These are bounded-buffer (problem-consumer) problem, the readers-writers problem and the dining philosophers problems. They demonstrate how shared resources can be managed safely while avoiding race conditions, deadlocks and starvation.

Bounded-buffer problem:- The bounded-buffer problem involves a producer that adds items to a limited size buffer and a consumer that removes them. Three semphores are used: mutex = 1 protect the buffer, empty = n counts free slots and full = 0 counts occupied slots.

Producer operations:- wait(empty), wait(mutex), add item, signal(mutex), signal(full).

Date .....
S M T W TH F S

CRYSTAL

Consumer operations:- wait(full), wait(mutex), remove item, signal(mutex), signal(empty).

Readers-writers problems:- The readers-writers problem involves a shared database that many readers can access simultaneously, while a writer need exclusive access. If reader always receive priority writers may starve; if writers receive priority, readers may starve. The solution uses tools such as mutex, a writer lock and a read_count variable. The first reader prevents writers from entering and the last reader allows writer to proceed.

Dining-philosophers problem:- Five philosophers sit around a table with five chopsticks one between each pair. Each philosopher alternates between thinking and eating and must acquire both neighboring chopsticks to eat. If all philosphers pick up their left chopstick simultaneously, each waits for right chopstick held by a neighbour. This creates deadlock because everyone waits indefinitely.

Solution to Dining-philosopher's problem:- Allow at most philosopher to eat at the same time, or let them pick up chopsticks only when both are available. Another solution is for odd and even philosophers to pick up different chopsticks first. These methods help prevent deadlock and starvation.

Monitors:- A monitor's is a synchronization tool that allows only one process or thread inside at a time. It uses `wait()` and `signal()` to control waiting and execution. Java `synchronized` keyword is an example of mutual exclusion.

CamScanner

Date .....
S M T W TH F S

CRYSTAL

Comparison of Synchronization tools:-

Tool	Main idea	Advantage	Limitation
Mutex	Lock with two states.	Simple protection	Has a specific owner.
Semaphore	Counter with wait and signal.	Control multiple resources.	Can be misused and caused deadlock.
Monitor	Language-level synchronization	Automatic mutual exclusion	Requires language support.
Spinlock.	Repeatedly checks a lock.	Fast for very waits.	Wastes CPU during busy waiting.